Alejandro Lucena
4/8/2026
COMP 468

Experiment 6: LeNet

Introduction and Methodology

This report evaluates the performance of a baseline LeNet-5 inference stack that implements a forward pass for grayscale images (e.g., MNIST) across varying batch sizes and cuDNN convolution algorithms—implicit_gemm, implicit_precomp, and fft. Output verification is done using a reference CPU implementation.

First, the platform details. All tests were conducted on an NVIDIA V100 GPU using CUDA version 12.2.1. Because GPU performance is highly dependent on hardware specifications, the GFLOP/s subsequently reported are specific to this GPU environment.

Next, a brief overview of the baseline implementation. The LeNet-5 inference stack begins with a convolution layer (C1) that transforms the 32x32 grayscale input into six 28x28 feature maps, followed by bias addition and Tanh activation. These maps are processed by a pooling layer (S2) using 2x2 average pooling with a stride of 2, reducing the dimensions to 14x14. A second convolution (C3) produces sixteen 10x10 maps, which are activated via Tanh (with bias addition) and then pooled again (S4) to a spatial size of 5x5. The resulting 16x5x5 (400 values) feature map is flattened and fed into a series of three fully connected layers. These layers use cuBLAS GEMM for matrix-vector multiplication, with bias addition and Tanh activations applied after the first two dense blocks. The final output is a 1D vector of 10 logits, representing the network's classification scores for the digits 0-9.

At a higher level, the convolutional layers act as feature extractors (finding edges, shapes, and patterns). The Fully Connected (FC) layers, on the other hand, act as a classifier, digesting all the features identified by the convolutions and deciding which digit it represents. The baseline implementation is agnostic to the chosen convolutional algorithm, enabling a performance study across the three aforementioned algorithms. To eliminate cold-start latency overheads, each implementation was run 100 times, and the reported results represent averages across all runs.

Finally, the performance metric. To quantify and compare throughput, this report utilizes GFLOP/s as its sole performance metric. For each convolution layer, the operation count per image is $CONV_OPS_{total} = (H_{out} \times W_{out} \times C_{out}) \times (C_{in}\ K_h, K_w) \times 2$. The

2 represents the two operations (Multiply + Add) performed for every weight. For each Fully Connected layer, the operation count per image is FC_OPS = Input Features × Output Features × Batch Size. Performance for LeNet is thus calculated as: GFLOP/s = (2 x $CONV_OPS_{total}$ + 3 x FC_OPS)/ Time (s) * 10^9, taking into account that FC_OPS is not the same across all three layers due to the image's dimensional reduction.

Performance and Memory Study

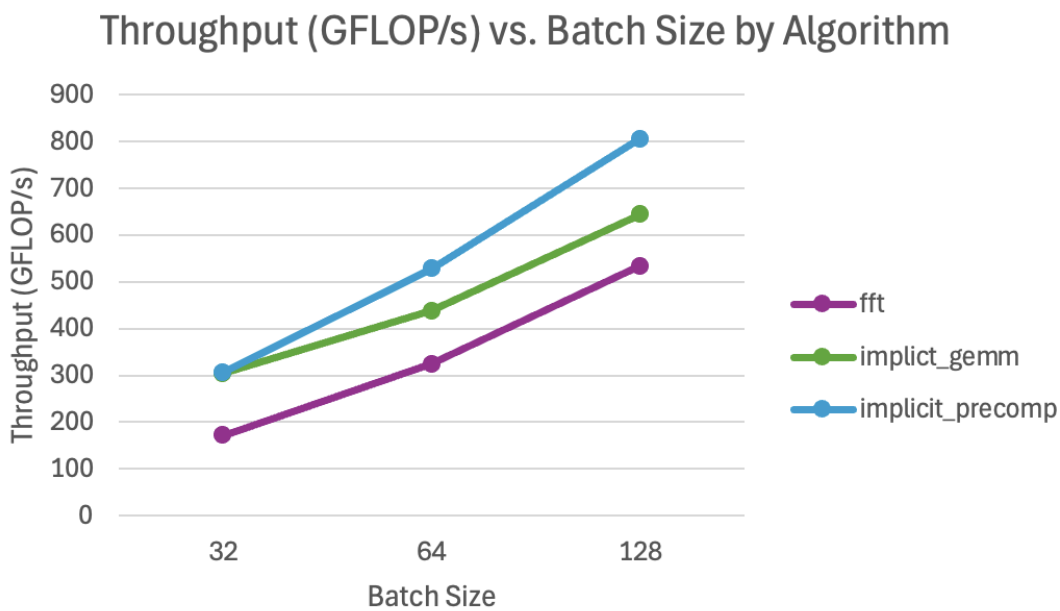

Figure 1

Figure 1 shows a scaling graph of increasing batch sizes {32, 64, 128} for each of the three aforementioned convolutional algorithms in the LeNet-5 baseline implementation. As shown in the figure, throughput (GFLOP/s) increases with increasing batch size. Notably, implicit_precomp is the most efficient convolutional algorithm, achieving the highest throughput of 805 GFLOP/s at a batch size of 128. Conversely, the Fast Fourier Transform (FFT) convolutional algorithm is the slowest, largely due to its higher overhead cost.

Implicit Precomp GEMM is often faster than Implicit GEMM because it precomputes and stores frequently used values (such as index offsets and filter-related data), thereby reducing redundant calculations. In contrast, Implicit GEMM performs these operations in an ad hoc manner, leading to lower reuse and thus lower efficiency.

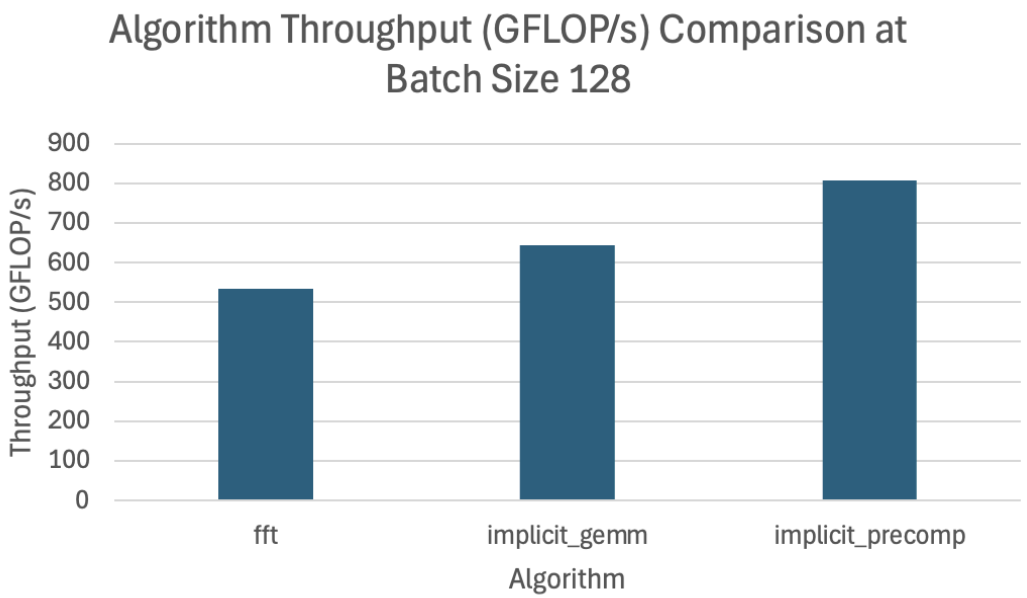

Figure 2

Figure 2 shows the throughput of the three algorithms mentioned above at a batch size of 128. Again, FFT is the slowest, achieving a throughput of 532.99 GFLOP/s. While FFTs are optimized for convolutions, the small spatial dimensions of MNIST (32x32) and LeNet (5x5) kernels do not provide enough work to overcome the transformation (signal-to-frequency) overhead. Consequently, implicit_precomp achieves ~1.5x higher throughput than FFT.

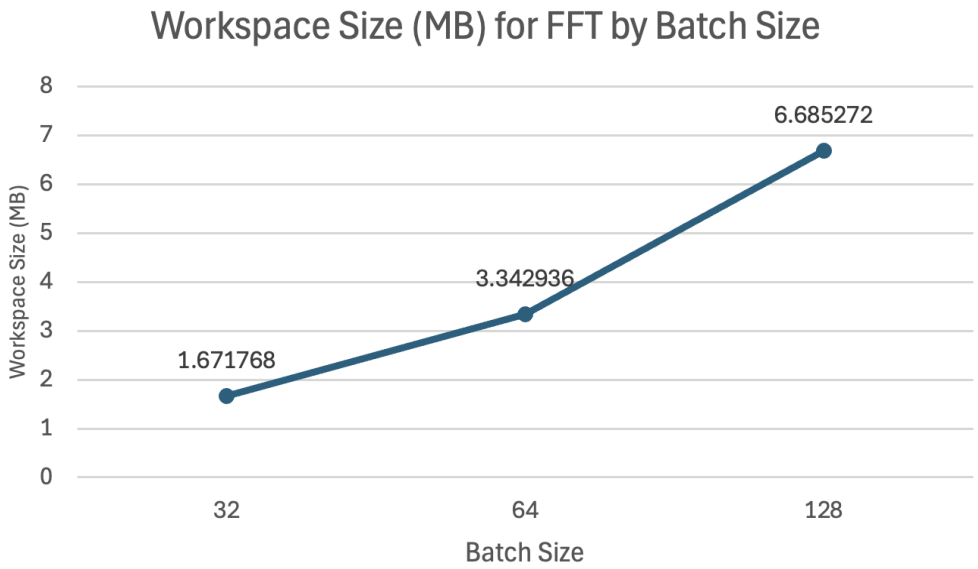

Figure 3

Figure 3 shows the workspace memory size (in MB) needed by the FFT convolutional algorithm across increasing batch sizes of 32, 64, and 128. As the figure shows, doubling the batch size roughly doubles the required workspace, because it needs to store a transform for every image on the batch, a 1-1 correspondence.

Curiously, both the implicit_gemm and implicit_precomp LeNet-5 implementations have a workspace size of 0 bytes. Implicit GEMM performs ad hoc calculations, so it has no need for a workspace. Conversely, Implicit Precomp GEMM typically uses a small workspace. Still, because of the small problem size, cuDNN often determines that this workspace can fit entirely in the GPU's shared memory or registers, eliminating the need for a global memory workspace. Consequently, the workspace for both convolutional algorithms remains zero bytes.

Key Takeaways and Conclusion

A limitation of this baseline LeNet-5 inference stack implementation is that it requires three separate function calls to run a single convolutional layer, regardless of the chosen convolutional algorithm: cudnnConvolutionForward, cudnnAddTensor (bias), and cudnnActivationForward. These separate passes necessitate "round-trip" synchronization with global memory for the bias and activation steps. Because memory bandwidth is a more constrained resource than computational throughput on this GPU, this creates a performance stall/bottleneck. Overloading the run_lenet_conv() kernel to call the single fused cudnnConvolutionBiasActivationForward would perform all three steps in a single kernel while keeping the data in the GPU’s fast registers, thereby achieving even higher throughput.

Ultimately, this experiment demonstrates that the convolutional algorithm matters when implementing the LeNet-5 inference stack. For larger problem sizes, FFT is the algorithm of choice, but for small problems like MNIST, other algorithms that perform convolutions directly, without any transformation overhead, are better choices. Furthermore, throughput scaled positively with batch size; increasing the batch size to 128 improved GPU occupancy and work saturation, allowing the hardware to amortize fixed launch overheads across more data.

In terms of performance, since the implementation relies on cuDNN's *implicit_gemm* kernel distribution and largely eliminates cold-start latency, it achieved near-peak theoretical throughput for the chosen cuDNN and cuBLAS methods. Finally, fidelity to the absolute error thresholds (<= 1e-3) was maintained relative to the CPU reference, as shown in the log below.

Log

```
[ajl18@bc11u25n1 exp6_LeNet]$ ./bin/dlenet --batch 32 --algo implicit_gemm --impl baseline
Max absolute difference: 9.31323e-09
Impl=baseline Batch=32 Algo=implicit_gemm Time(ms)=0.09 GFLOP/s=305.65 Workspace_bytes=0
[ajl18@bc11u25n1 exp6_LeNet]$ ./bin/dlenet --batch 128 --algo implicit_gemm --impl baseline
Max absolute difference: 9.31323e-09
Impl=baseline Batch=128 Algo=implicit_gemm Time(ms)=0.16 GFLOP/s=647.74 Workspace_bytes=0
[ajl18@bc11u25n1 exp6_LeNet]$
```